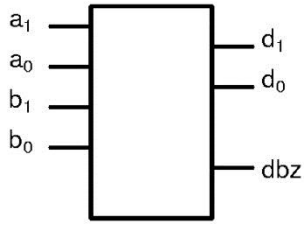
25.5.2018

EE 241 DIGITAL CIRCUITS
FINAL SOLUTION MANUAL

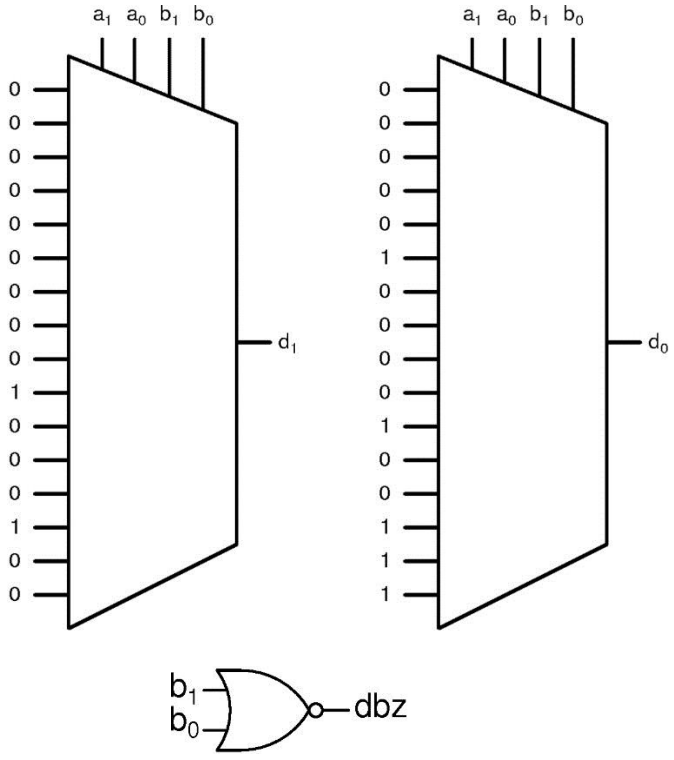
Name:

Q1) (15 points) Design a digital system (as a combinational circuit) to divide two numbers using multiplexers only. Inputs to the system are a_1a_0 and b_1b_0 which are two-bit numbers. Division is defined as d_1d_0 which is also a two-bit number. Your system should also indicate when a division by zero occurs. When this case happens, d_1d_0 should be logic level zero and output pin (dbz) should be logic level one. Design the system using **only two multiplexers** for the division operation and **sufficient number of logic gates** for the dbz output.

Note: Verilog description of the system is NOT required in this question.



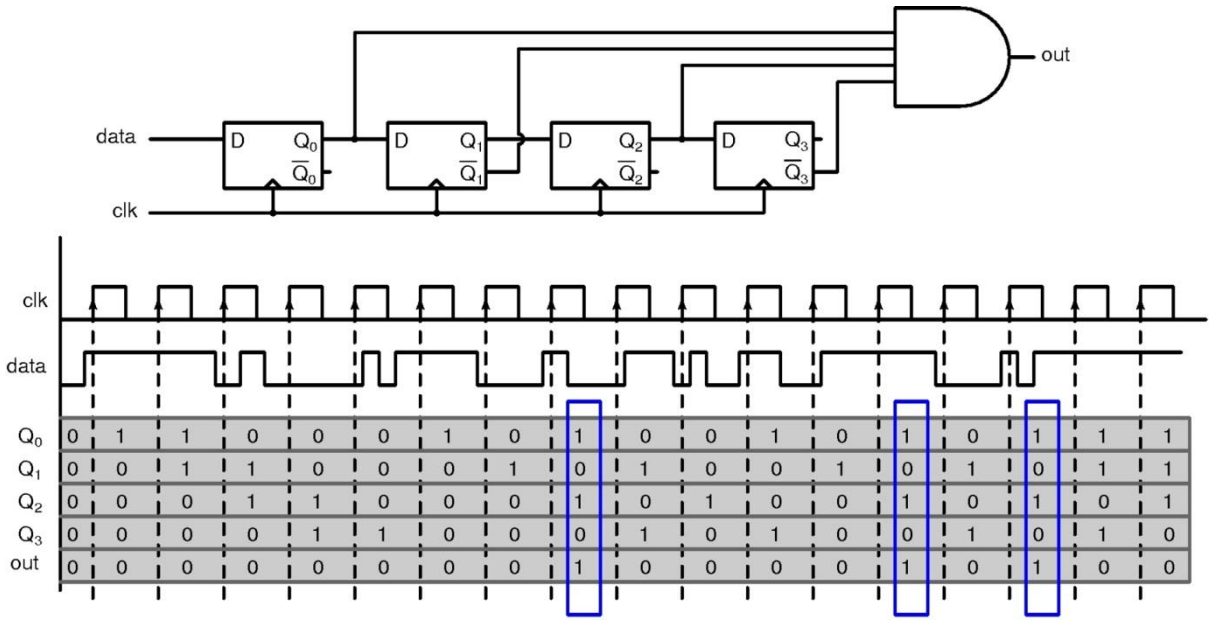
Solution:



Q2) (15 points) A sequence detector circuit is implemented by rising-edge-triggered flip-flops. Using the schematic and timing diagram given below, fill in the table for the outputs **Q_0, Q_1, Q_2, Q_3,** and **out**.

Note: This is a binary question such that each row below gets 3 points when correct.

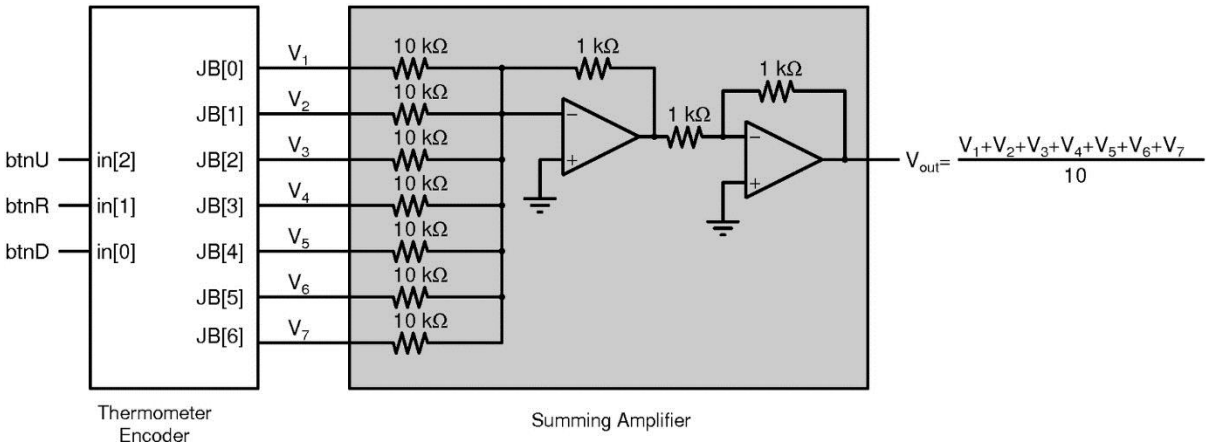
Solution:



Q3) (35 points) Digital-to-Analog Converter (DAC) converts a given digital value to analog form. Such a structure is given below. Within this system, three-bit input (in[2:0]) is fed to **the thermometer encoder module**. Output of this module is connected to the **analog summing amplifier module** to create the corresponding analog output voltage. To note here, when any output of the thermometer encoder module is set to logic level 1, it will have 3.3 V otherwise, its value will be 0 V. You are not responsible for the design of the analog summing amplifier module. It has been done beforehand as given below. You are asked to design the thermometer encoder module and implement it on the Basys3 board using Verilog. The thermometer encoder module should work such that based on the binary input value successive output pins should be set to logic level one. Let's give four examples.

i. When the input to the module is in=3'b000, its output will be JB=7'b0000000;
ii. When the input to the module is in=3'b001, its output will be JB=7'b0000001;
iii. When the input to the module is in=3'b010, its output will be JB=7'b0000011;
iv. When the input to the module is in=3'b011, its output will be JB=7'b0000111;

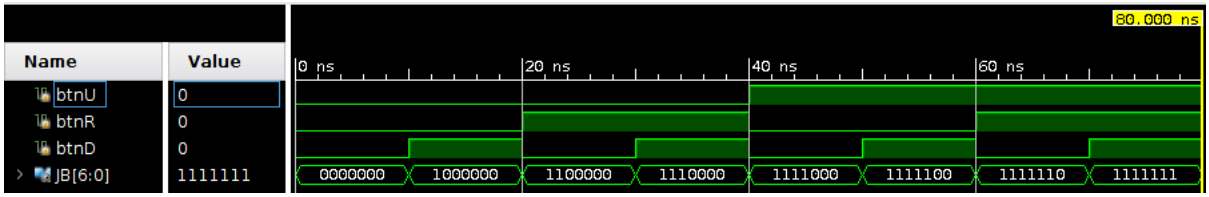
and so on.



Solution:

```
`timescale 1ns / 1ps
module main(btnU,btnR,btnD,JB);
input btnU,btnR,btnD;
output reg [6:0] JB;
always @(*)
   case ({btnU,btnR,btnD})
   0: JB=7'b0000000;
   1: JB=7'b1000000;
   2: JB=7'b1100000;
   3: JB=7'b1110000;
   4: JB=7'b1111000;
   5: JB=7'b1111100;
   6: JB=7'b1111110;
   7: JB=7'b1111111;
   endcase
endmodule
```

```
module main_tb;
reg btnU,btnR,btnD;
wire [6:0] JB;
main u1(btnU,btnR,btnD,JB);
initial begin
{btnU,btnR,btnD}=3'd0;
repeat (8)
#10 {btnU,btnR,btnD}={btnU,btnR,btnD}+1'b1;
$stop;
end
endmodule
```


Name
Value
btnU
0
btnR
0
btnD
0
JB[6:0]
1111111
0 ns
20 ns
40 ns
60 ns
80.000 ns
0000000
1000000
1100000
1110000
1111000
1111100
1111110
1111111

Q4) (35 points) The Fibonacci sequence is defined by the relation $F_{n+2}=F_{n+1}+F_n$ where $F_0=0$ and $F_1=1$. Using this relation, the Fibonacci sequence can be obtained as 0, 1, 1, 2, 3, 5, 8, 13, …

Implement the Fibonacci Number Generator module using the Basys3 Board with the following specifications. Everytime the center button (btnC) is pressed; the new number is generated and displayed on the 4-digit seven segment display. When the maximum display value is exceeded, the module resets itself. To make the system more modular, represent the parts of the module with below submodules.

a) **Clock Division Module:** This module is needed since there is an upper limit (1 kHz) for the clock frequency while performing a scanning operation on the four digits of the seven segment display. Since the human eye can detect flickers around 30 Hz, design a clock divider which divides the master clock of Basys3 Board (100 MHz) to some value between 30 Hz and 1 kHz. Let's pick this value as **500** Hz. The layout of the module is as follows.

```
module clkdiv(clkin,clkdiv);
input clkin;
output  clkdiv;
```

b) **7-Segment Converter Module:** Since each number is displayed on four separate 7-segment display, a converter module is needed. The layout of the module is as follows.

```
module sevseg(in,out);
input [3:0] in;
output [6:0] out;
```

c) **Scanning Display Controller Module:** This module is needed to display four digits of the Fibonacci number on the four digits of the seven segment display. This module displays the Fibonacci digits by turning on and off separate 7-segments. Once the divided clock and 7-Segment converter module is ready, a module is needed for separating a number's digits into thousands, hundreds, tens and ones. This representation is shown on the four digit 7-segment display. The layout of the module is as follows.
Note: This module is in fact provides a binary decimal coded output.

```
module scan(clkdiv,an,seg,number);
input clkdiv;
input  [12:0]  number;
output [6:0] seg;
output reg [3:0] an;
```

d) **The Fibonacci Number Generator Module:** Instantiate all the necessary modules you have created above. Make sure that display resets itself when the value of the number exceeds 5000**.** The layout of the module is as follows.

```
module fibo(clk,btnC,seg,an);
input clk,btnC;
output [6:0] seg;
output [3:0] an;
```

Solution:

a)

```
`timescale 1ns / 1ps
module clkdiv(clkin,clkout);
input clkin;
output reg clkout;
reg [16:0] A=17'd0;

always @(posedge clkin)
if(A==17'd99999)
   begin
      A<=17'd0;
      clkout <= ~clkout;
   end
   else
      A<=A+1'b1;
endmodule
```

b)

```
`timescale 1ns / 1ps
module sevseg(in,out);
input [3:0] in;
output reg [6:0] out;
always @(*)
begin
case(in)    //gfedcba//
   0:  out = 7'b1000000; // 0
   1:  out = 7'b1111001; // 1
   2:  out = 7'b0100100; // 2
   3:  out = 7'b0110000; // 3
   4:  out = 7'b0011001; // 4
   5:  out = 7'b0010010; // 5
   6:  out = 7'b0000010; // 6
   7:  out = 7'b1111000; // 7
   8:  out = 7'b0000000; // 8
   9:  out = 7'b0010000; // 9
endcase
end
endmodule
```

c)

```
`timescale 1ns / 1ps

module scan(clkin,an,seg,number);
input clkin;
input [12:0] number;
output [6:0] seg;
output reg [3:0] an;

reg [1:0] cnt=2'b00;
reg [3:0] segin;
wire [3:0] tho,hun,ten,one;

sevseg u1(segin,seg);

assign tho=number/1000;
assign hun=(number%1000)/100;
assign ten=((number%1000)%100)/10;
assign one=((number%1000)%100)%10;

always @(posedge clkin)
   begin
   cnt=cnt+1'b1;
   case (cnt)
   0:  {an,segin}={4'b0111,tho};
   1:  {an,segin}={4'b1011,hun};
   2:  {an,segin}={4'b1101,ten};
   3:  {an,segin}={4'b1110,one};
   endcase
   end
endmodule
```

d)

```
`timescale 1ns / 1ps
module fibo(clk,btnC,seg,an);
input clk,btnC;
output [6:0] seg;
output [3:0] an;
reg btn;
reg [12:0] a=13'd0,b=13'd1,sum=13'd0;
wire clkd;

clkdiv u1(clk,clkd);
scan u2(clkd,an,seg,sum);

//registering the input btnC to btn is essential for implementation
always @(posedge clk)
btn = btnC;

always @(posedge btn) //btnC is replaced by btn for implementation
begin
  if (sum>5000)
    begin
    a=0;
    b=1;
    sum=0;
    end
  else
    begin
    sum=a+b;
    a=b;
    b=sum;
    end
end
endmodule
```